What a Pooled Leaderboard Does Not Say: Auditing the Scope and Provenance of Method Comparisons in Microservice RCA Benchmarks

LINING HU, Shanghai Jiao Tong University, China
TING LIU, Shanghai Jiao Tong University, China
YUZHUO FU*, Shanghai Jiao Tong University, China

arXiv:2606.29159v2 [cs.AI] 13 Oct 2026

Offline benchmarks for microservice root cause analysis (RCA) rank methods by a score pooled over several applications, and a pooled lead leaves two things unstated: on which of the systems it holds, and how the score was produced. We audit both on three public benchmark families (OpenRCA, RCAEval, and PetShop), running the released implementations of 16 methods under recorded configurations on 11 subsystems and on a second release of three of the applications. Whether a pooled lead between published methods holds on every subsystem is decided by a quantity the leaderboard withholds, the spread of the lead across subsystems: of 15 leads whose margin exceeds this spread, 14 hold everywhere; of the 8 whose margin is below it, 6 reverse. The per-fault and per-scenario tables that the benchmark papers publish split the same way, and every error of a ranking computed without one subsystem falls on a lead of the second kind. On the second release a third of the comparisons change sign, most through one method's response to the metric kinds it records, and on several subsystems a simple probe or a shipped baseline beats every published method. How the score was produced matters as much: the input table that one released runner reads is not the one its documentation describes, and this choice alone reverses a comparison between published methods with intervals excluding zero both ways; an unpinned dependency and a duplicated column header each make a method return its input column order without an error. Of 23 RCA papers we examined, none releases the per-case outputs needed to check either premise. We recommend that a pooled comparison state the subsystems it covers and its spread across them, the input table, the handling of failed outputs, and the denominator, and be accompanied by per-case outputs, as ours are in our replication package.

1 Introduction

When an offline benchmark for microservice root cause analysis (RCA) reports that method *A* outperforms method *B*, what does the claim apply to? Benchmarks such as OpenRCA [38], RCAEval [26], and PetShop [9] each contain several applications or traffic scenarios, which we call subsystems, and methods are ranked by a score pooled over them. A pooled comparison can be valid for the evaluated mixture without establishing an advantage on each constituent system; yet a method is adopted for one system, and a pooled ranking is read as a guide to that choice. The number on the leaderboard does not say whether the advantage holds on the system at hand. It leaves two premises unstated: on which systems the advantage holds, and how the score was produced. Figure 1 shows one instance of each. Two pooled leads of similar size (a) differ once split by subsystem (b): one has the same sign on every application, the other is reversed on one scenario with a confidence interval (CI) that excludes zero, and what separates them is the spread of the lead across subsystems, which the leaderboard does not report. On the same cases, reading the input table that the released runner selects instead of the one its documentation describes (c) reverses a comparison between two published methods, with CIs excluding zero both ways.

Three observations motivate a systematic check. Papers report per-system results but state superiority over the whole benchmark, and the per-case outputs needed to check such a claim with

*Corresponding author.

Authors' Contact Information: Lining Hu, ring.hu@sjtu.edu.cn, School of Electronic Information and Electrical Engineering, Shanghai Jiao Tong University, Shanghai, China; Ting Liu, louisa_liu@sjtu.edu.cn, School of Electronic Information and Electrical Engineering, Shanghai Jiao Tong University, Shanghai, China; Yuzhuo Fu, yzfu@sjtu.edu.cn, School of Electronic Information and Electrical Engineering, Shanghai Jiao Tong University, Shanghai, China.

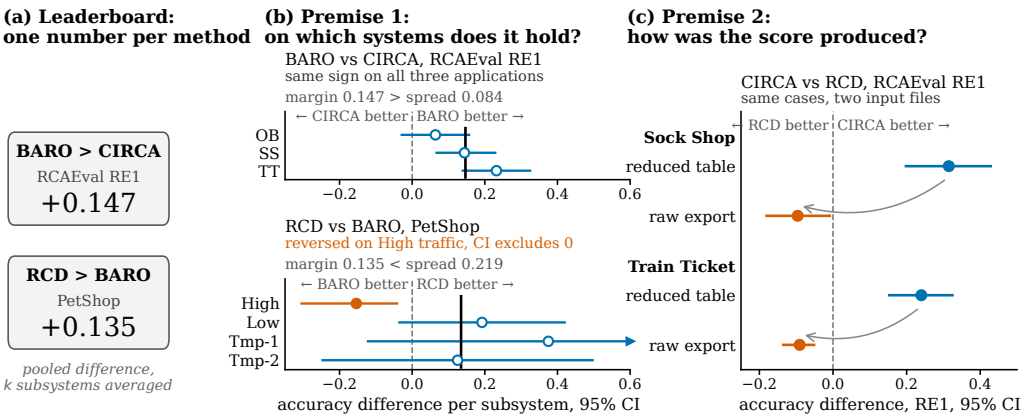

Fig. 1. (a) Two pooled comparisons of similar size from two benchmarks. (b) The same comparisons split by subsystem (OB, SS, TT: RCAEval's Online Boutique, Sock Shop, and Train Ticket): points are subsystem differences Δ_s with 95% bootstrap CIs, the vertical bar is the pooled difference, labels name the method each side favours, orange marks a subsystem whose sign is opposite to the pooled one, filled when its CI excludes 0, and each group states its pooled margin next to the between-subsystem spread of Δ_s. (c) On the same RCAEval cases, the two input tables that the release ships reverse CIRCA vs RCD on two applications, with CIs excluding 0 under both.

its uncertainty are rarely released. The releases leave choices to whoever runs them, and some of these choices change scores without raising an error: RCAEval's runner reads the one of its two metric tables that its documentation does not describe, one method returns its input column order under a newer version of a dependency the release does not pin, and a duplicated column header in 50 input tables makes another method fail in the same silent way. Recent work asks whether these benchmarks are hard enough [8], but not which of the comparisons drawn from them hold across their subsystems.

We audit three public benchmark families: 11 subsystems in the main analysis, and a second, independently collected release of three of the applications. We run the released implementations of 16 methods (10 published methods, 3 simple probes, and 3 baselines shipped by a benchmark) under recorded configurations and score them case by case on every case of the subsystems where they run. We compare these runs, not the numbers the original papers report; on OpenRCA, where no published comparison of these methods exists, the comparison is ours (Section 3.1). We ask one question about each premise. **RQ1 (scope):** how stable are comparisons between published methods across the subsystems of a benchmark, and across a second release of the same applications? **RQ2 (provenance):** which choices in the released pipeline—the input representation, the handling of failed outputs, the denominator, and the scoring and weighting rule—change these comparisons, and by how much?

Our findings are:

- **Within a benchmark, a pooled lead between published methods holds on every subsystem when its margin exceeds the spread of the lead across subsystems, and mostly reverses when it does not; the leaderboard reports the margin and withholds the spread.** Of 15 such leads with margin above spread, 14 hold on every subsystem; of the 8 below it, 6 reverse, among them both reversals with CI support. With probes and shipped baselines the counts are 25 of 34 against 2 of 64, and in the per-fault and per-scenario

tables the benchmark papers publish, 27 of 31 against 2 of 39. A ranking computed on all subsystems but one picks the less accurate of two adjacent methods on the held-out subsystem in 8 of 36 decisions, every one on a lead whose margin is below its spread. On several subsystems a probe, or a baseline shipped by the benchmark itself, is more accurate than every published method.

- **How the score was produced can reverse it.** The largest change between the two releases of the same applications is one method's response to the metric kinds the second release records: of 18 comparisons repeated on it, 6 change sign, 5 of them involving BARO, which ranks a disk I/O metric first that the first release does not contain. On the same cases, the input table that the released runner reads reverses CIRCA against RCD on two applications, with CIs excluding zero in both directions. Two failures leave no trace on the leaderboard. Under a drifted dependency one method returns its input column order on every case. A duplicated column header makes another method do the same, and produces the only CI-supported reversal between published methods on RCAEval's first release; it disappears once the header is removed. The denominator, the fault composition, and the scoring rule move only differences that are already close to zero.
- **Published papers do not release what is needed to check either premise.** Of 23 RCA papers we examined (Section 6), every one that evaluates more than one system reports per-system results. 7 release per-case outputs of at least one method, 3 of them only for a single demonstration case. None of the 21 whose releases we could reach releases enough to recompute every paired comparison of its main results without rerunning the methods.

This paper contributes: (i) a matched per-case table over three RCA benchmark families and 16 methods, with the configuration of every run, and two integrity checks for released pipelines that anyone can apply: whether the runner reads the input table that the documentation describes, and whether a method's ranking equals its input column order; (ii) a quantification of the two unstated premises, the scope of pooled comparisons across subsystems, a second release, and held-out subsystems, and the provenance of scores across the recorded metric kinds and four stages of the released pipelines; and (iii) a reporting format that states both premises, with recommendations to benchmark maintainers tied to the problems we verified. We do not propose an RCA method.

2 Background and Research Questions

2.1 Offline RCA Benchmarks

In microservice root cause analysis, a method receives telemetry from a window around a failure (metrics and, in some benchmarks, logs and traces) and returns a ranked list of candidate root causes, usually services or components. A case is scored by whether the true root cause is ranked first (Acc@1) or, in OpenRCA, by the fraction of required answer items that are correct. We audit offline benchmarks, which release fault cases, ground truth, and a scoring rule so that every method can be scored on the same cases (OpenRCA [38], RCAEval [26], PetShop [9]); single-application testbeds such as Train Ticket [49], competitions, and interactive benchmarks in which an agent queries a live system or a recorded snapshot through diagnostic tools [3, 13, 35] are outside its scope.

An offline benchmark is released as per-case data tables, a runner that loads them and executes the methods, and a scoring script. A pooled score is the product of that pipeline: which table the runner reads, what happens when a method fails on a case, which cases enter the denominator, and how cases are scored and weighted. The benchmark papers state the scoring rule; the other choices are made in code. Each benchmark also contains several applications or traffic scenarios,

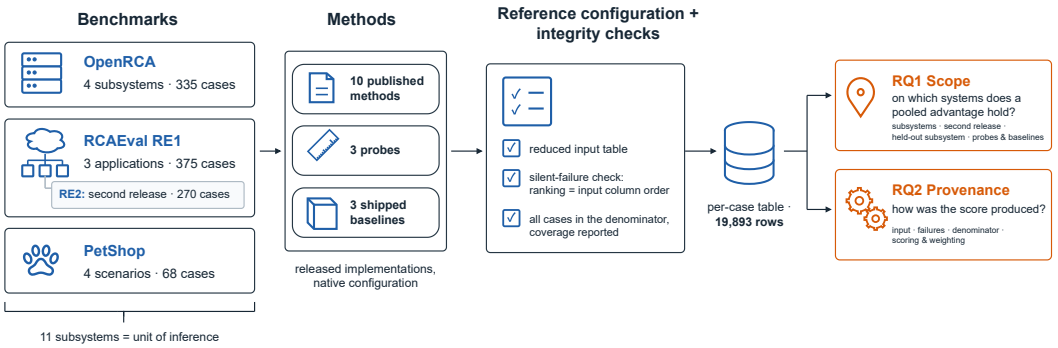

Fig. 2. Overview of the audit. Methods are scored case by case on every subsystem where they run; RQ1 asks on which subsystems a pooled advantage holds, RQ2 which stages of the released evaluation pipeline change it.

which we call subsystems; its paper and the method papers that use it report a score per subsystem and a score pooled over all of them, and it is the pooled score that is ranked.

2.2 Research Questions

A pooled leaderboard reports one number per method. Reading it as evidence that method A should be preferred to method B assumes two things the number does not state: on which systems the advantage holds, and how the score was produced. We ask one question about each. **RQ1 (scope):** how stable are comparisons between published methods across the subsystems of a benchmark, and across a second, independently collected release of the same applications? **RQ2 (provenance):** which choices in the released evaluation pipeline—the input representation, the handling of failed outputs, the denominator, and the scoring and weighting rule—change these comparisons, and by how much?

3 Audit Design

Figure 2 summarizes the audit: three benchmark families with eleven subsystems and a second release of three of them, sixteen methods in three roles, one reference configuration with integrity checks, and the two questions asked of the resulting per-case table.

3.1 Benchmarks and Subsystems

We audit three public benchmark families; Table 1 lists their subsystems with the number of cases of each. A *subsystem* is one application or scenario inside a family. It is the unit at which an operator would adopt a method, and it is our unit of comparison.

OpenRCA [38] contains four subsystems (Bank, Market-1, Market-2, Telecom). Each query asks for one or more of the root-cause component, the failure reason, and the occurrence time; the official scorer awards the fraction of the required items that match (component and reason by exact string, time within 60 s). The audited methods rank metrics, so we wrap them. A method ranks the metrics of the query window (a 4-hour baseline plus 30 minutes on either side of the window midpoint). The answer component is the first ranked metric that belongs to the query's candidate set, the reason is mapped from that metric's type, and the time is the moment of largest deviation. For a query that states n failures, the wrapper returns the first n distinct candidate components. No published work compares these methods on OpenRCA, whose own leaderboard ranks LLM agents; the comparison is our construction through this wrapper, and RQ1 marks it as such. CD-1min runs

on one-minute data with a detection window that ends one minute after the query midpoint, the setting for which it was written.

RCAEval [26] releases three suites over the same three applications (Online Boutique, Sock Shop, Train Ticket). RE1 (metrics only; five fault types) provides three subsystems of the main analysis. RE2 (six fault types; metrics, logs, and traces) is a second, independently collected release of the same applications; we use it only to ask whether conclusions carry over between releases, and do not count it as further subsystems. The scoring rule is top-1 at service level, a ranked metric mapping to the service named by its prefix, with the benchmark's default window of 20 minutes around the injection time.

PetShop [9] provides four traffic scenarios (high traffic, low traffic, and two temporal scenarios), scored by its official harness at node level.

The main analysis thus has $k = 11$ subsystems and 778 cases, and each method is scored on every case of the subsystems where it runs; RE2 adds 270 cases.

3.2 Methods and Their Roles

Methods enter the audit in one of three roles. *Published methods* are the objects of the audit: BARO [24], CIRCA [17], ϵ-Diagnosis [32], RCD [11], and CausalRCA [37] on RE1; on RE2 additionally SimpleRCA [8], TraceRCA [18], MicroRank [40], and the multi-source variant of BARO shipped with RCAEval; on OpenRCA BARO, ϵ-Diagnosis, and RCD; on PetShop CIRCA, RCD, ϵ-Diagnosis, counterfactual attribution [2], and BARO. We run the implementations that the benchmarks ship, at the release current when the audit began; the version is recorded in the replication package. For SimpleRCA we use the RCAEval branch of the authors' released code, which reproduces to two decimals every value the authors report for SimpleRCA and their BARO baseline on RE2 and on RE3, a third release of the same applications that we do not otherwise use. On RE1, which provides only metrics, that branch reduces to its BARO-based metric ranking, so we do not count it as a separate method there.

Probes are three simple rules that we add to test whether a leaderboard orders even simple alternatives consistently. max-$|Z|$ picks the metric with the largest absolute z-score after the fault relative to the pre-fault interval; alert-count picks the service with the most metrics whose z-score exceeds 3; CD-1min, a threshold detector written for OpenRCA at one-minute resolution, flags metrics whose post-onset peak exceeds the baseline in both z-score and relative change and ranks components by their largest relative change. *Shipped baselines* are the traversal, ranked-correlation, and random-walk baselines that PetShop distributes.

Not every method runs on every family: MicroRank and TraceRCA need traces, which only RE2-OB and RE2-TT provide; CausalRCA would need an estimated 4,900 GPU-hours on OpenRCA; CIRCA's causal-discovery step fails on OpenRCA because the metric correlation matrix is singular when there are more metrics than time points, and making it run would mean choosing a metric subset; and MicroCause, also shipped with RCAEval, took more than 10 minutes per case and was not run. RCD and CausalRCA are randomized; we run three seeds and average each case's score over seeds, except for the RCD implementation that PetShop ships, which we run once through its harness.

3.3 Reference Configuration and Integrity Checks

A released benchmark can be run in more than one way that is consistent with its documentation, and some of those ways change scores without any visible error. Before comparing methods we therefore fix a reference configuration and check the outputs it produces; the alternatives set aside here become the conditions of RQ2.

Input representation. RE1-SS and RE1-TT ship two metric tables per case: a raw export with several hundred to over a thousand columns, and a reduced table with a few dozen to about two hundred.[1] The released runner reads the raw export, and the code that would switch to the reduced table is commented out; the metric counts in the repository's documentation, however, match the reduced table. The choice matters: BARO's Avg@5 (the metric its authors report) on the reduced tables is 0.960 (SS) and 0.779 (TT), close to the 0.95 and 0.81 that its authors, who also built RCAEval, report, but 0.576 and 0.261 on the raw exports. BARO is the only method whose reported values identify the file in this way, so we take the reduced tables as the reference on these two grounds; the raw exports remain an equally literal reading of the runner, and Section 5.1 compares every method's outputs on both.

Silent failures. A method that fails inside the pipeline may still return a complete ranking, and nothing in the output marks it. We check for the simplest signature of such a failure, a ranking equal to the input column order, and found two sources of it in the audited release. The first is dependency drift: CausalRCA calls a PageRank function that scikit-network removed in version 0.32, the released requirements set only a lower bound on that library, and the resulting exception is caught and the input column order returned. Under scikit-network 0.33.0, CausalRCA's ranking equalled the column order in 30 of 30 cases of a permutation test; under 0.31.0 in 0 of 30, and we run it under that version. The second is a duplicated column header in 50 of the 125 RE1-OB tables, on which CIRCA fails and returns the column order, and ϵ-Diagnosis ranks the duplicated column first, in 49 cases each. Both are fixed in the current upstream release. We apply the same fix to the input, rerun the affected methods on the affected cases, and keep the as-shipped outputs for Section 5.2.

Conventions. All RE1 and RE2 methods use the 20-minute default window. An output is invalid if the method raised an error, returned no candidate, or, on OpenRCA, the wrapper fell back to a fixed default answer. Invalid outputs score 0 and the denominator is always the full case set. We mark every accuracy whose coverage (the share of valid outputs) is below 1 and report the coverage values in Section 5.3 and the replication package. All per-case outputs, each with the configuration of the run that produced it, are merged into one table of 19,893 rows; after freezing, every analysis reads only this table.

3.4 Statistics

For methods A and B on subsystem s with n_s matched cases, the subsystem effect is the mean paired difference $\Delta_s = n_s^{-1} \sum_i (y^A_{s,i} - y^B_{s,i})$, where y is the case score (top-1 hit, or the OpenRCA partial score). Its 95% confidence interval (CI) is a percentile bootstrap over cases within the subsystem ($B = 10{,}000$) [7]. The *pooled* difference $\bar{\Delta}$ is the unweighted mean of Δ_s over the k subsystems of a family, so that each subsystem counts once, and its CI averages the subsystems' bootstrap replicates with the same weights; weighting by cases instead changes the reversal count of 1 pair only.

A *point-estimate reversal* is a subsystem whose Δ_s has the opposite sign of a non-zero pooled difference; a *CI-supported reversal* additionally has its CI entirely on the opposite side of zero. The pooled difference itself need not exclude zero: a CI-supported reversal says that one subsystem has a nominal 95% CI entirely on the side opposite to the pooled point estimate. The CIs are nominal and not adjusted for the number of comparisons (Section 9). A *sign change* between two conditions includes a difference that becomes or stops being zero. We also describe each comparison by its *pooled margin* $|\bar{\Delta}|$ and its *between-subsystem spread*, the sample standard deviation of Δ_s over

[1] The raw export is `data.csv` (421–439 columns on SS, 1,180–1,446 on TT) and the reduced table `simple_data.csv` (58–64 and 199–239 columns). The two tables of a case cover the same time points; the reduced table keeps a few aggregated metrics per service (CPU, memory, workload, latency percentiles, errors), the raw export the per-container series they are derived from. RE1-OB ships a single table of 51–60 columns, which we use. RE2 ships the reduced table `simple_metrics.csv`.

the k subsystems. The replication package also gives, for every pair, a random-effects prediction interval [12] where $k \geq 4$ and a case-level test of the method-by-subsystem interaction; with $k \leq 4$ the interval rests on two degrees of freedom and almost always includes zero, so we draw no conclusion from it.

Held-out subsystems. To ask whether a pooled ranking chooses correctly on a subsystem it has not seen, we hold out each subsystem of a family in turn and rank the methods by their mean over the remaining subsystems. For every adjacent pair in that ranking we check whether the higher-ranked method is also more accurate on the held-out subsystem. The *regret* of a pooled winner on subsystem s is the accuracy of the best method on s minus the winner's accuracy there.

Cross-family set. The methods with outputs on all 11 subsystems, BARO, RCD, ϵ-Diagnosis, and the three probes (6 methods, 15 pairs; on PetShop, RCD and ϵ-Diagnosis are the implementations PetShop ships), are also compared across families. Because OpenRCA's partial credit and top-1 accuracy are on different scales, we do not average them: we ask only whether a pair's subsystem differences have both signs, and whether that remains so when a family or two subsystems are dropped.

Throughout, we report two layers: comparisons between published methods, which are what a leaderboard is read for, and all pairs including probes and shipped baselines. The former compare the released implementations as we ran them.

3.5 Use of LLM Tools

LLM sessions were used to locate the tables, passages, and repository contents behind the checks of Section 6; every statement made there was verified by an author against the paper or the repository. No LLM produced or scored a method output.

4 RQ1 (Scope): On Which Systems Does a Pooled Advantage Hold?

Table 1 gives every method's accuracy on every subsystem; cells with coverage below 1 are marked, and randomized methods are averaged over three seeds. Within a family the best method changes from column to column: on RE1 it is BARO on two applications and CD-1min on the third; on PetShop it is a shipped baseline in all four scenarios, but not always the same one. RQ1 asks how often a comparison that a pooled leaderboard would report holds on each of the subsystems behind it.

4.1 Comparisons Between Published Methods

Figure 4 shows every comparison between two published methods that a family allows, with each subsystem's difference and its CI: 23 pairs, 10 on RE1, 10 on PetShop, and 3 on OpenRCA, where the comparison is ours (Section 3.1). 10 of them involve ϵ-Diagnosis, which is never more accurate than the other method on any subsystem, and none of these reverses. Of the 23 pairs, 7 reverse on at least one subsystem and 2 do so with CI support, both on one PetShop scenario (on RE1 and PetShop alone, 6 and 2 of 20). Which pairs reverse is decided by the two quantities of Section 3.4, the pooled margin and the between-subsystem spread of Δ_s (Fig. 3). Of the 15 pairs whose margin exceeds the spread, 14 hold on every subsystem; the exception is CIRCA vs counterfactual attribution on PetShop, reversed on the high-traffic scenario by -0.038 $[-0.269, 0.192]$. Of the 8 pairs whose margin is below the spread, 6 reverse, among them both CI-supported reversals. With probes and shipped baselines, 25 of the 34 pairs below the line reverse and 2 of the 64 above it. The two tables in which the benchmark papers themselves split a score by stratum, RCAEval's RE2-TT accuracy by fault type and PetShop's accuracy by scenario and fault type, split the same way: 27 of 31 pairs with margin below spread reverse on some stratum, against 2 of 39 above it. A leaderboard reports the margin; the spread is what it withholds. Dropping the two PetShop scenarios with 8 cases leaves

Table 1. Accuracy (Acc@1; OpenRCA: official partial score) of every method on every subsystem, over all cases (invalid outputs score 0). †: coverage below 1. Bold: best in column. –: the method does not run there (Section 3.2). RCD and CausalRCA: mean of three seeds (PetShop's own RCD: one run).

	RCAEval RE1			RCAEval RE2		
n	125	125	125	90	90	90
Method	OB	SS	TT	OB	SS	TT
Published methods						
BARO	**0.736**	0.848	**0.560**	0.144	0.067	0.667
CIRCA	0.672	0.704	0.328	0.667	0.778	0.544
ϵ-Diagnosis	0.032	0.144	0.000	0.033	0.156	0.000
RCD	0.541	0.389	0.088	–	–	–
CausalRCA	0.616	0.176	0.136	0.207	0.148	0.207†
SimpleRCA	–	–	–	0.467	0.244	**0.800**
TraceRCA	–	–	–	0.100	–	0.533†
MicroRank	–	–	–	0.000	–	0.122†
BARO (multi-source)	–	–	–	0.167	0.067	0.689†
Probes						
max-$\lvert Z\rvert$	0.504	0.744	0.424	**0.778**	0.844	0.411
Alert-count	0.392	0.376	0.048	0.211	0.389	0.211
CD-1min	0.728	**0.928**	0.280	0.733	**0.944**	0.789

	OpenRCA				PetShop			
n	136	70	78	51	26	26	8	8
Method	Bank	Mkt-1	Mkt-2	Tel	High	Low	Tmp-1	Tmp-2
Published methods								
BARO	0.160	0.057	0.106	0.183	0.154	0.154	0.250	0.250
CIRCA	–	–	–	–	0.269	0.385	0.500	0.625
ϵ-Diagnosis	0.036	0.038	0.044	0.046†	0.000†	0.000†	0.000†	0.125
RCD	0.089	0.051†	0.066†	0.193†	0.000	0.346	0.625	0.375
Counterfactual	–	–	–	–	0.308	0.154	0.125	0.250
Probes								
max-$\lvert Z\rvert$	0.127	0.161	**0.200**	0.232	0.077	0.192	0.375	0.375
Alert-count	**0.172**	0.144	0.197	**0.242**	0.038	0.000	0.000	0.250
CD-1min	0.165†	**0.161**	0.192	0.163†	0.000	0.038	0.125	0.125
Shipped baselines								
Traversal	–	–	–	–	0.385†	**0.577**†	0.625	**0.875**
Ranked correlation	–	–	–	–	**0.731**†	**0.577**†	**0.750**	0.500
Random walk	–	–	–	–	0.000†	0.000†	0.125	0.000

5 reversing pairs, 1 with CI support, and at a Bonferroni-adjusted level 1 CI-supported reversal remains (Section 9).

RCAEval RE1. Of 10 pairs, 1 reverses, without CI support. BARO leads each of the other four published methods on all three applications, with pooled margins from 0.147 to 0.656. The one unstable pair is CausalRCA vs RCD: its pooled difference is −0.030 [−0.077, 0.017], made of +0.075 on OB, −0.213 on SS, and +0.048 on TT; both methods are randomized, and the pooled sign itself changes with the seed (+0.027, −0.035, −0.083). This is RE1 under the reference configuration; on the files as shipped it is less stable (Section 5).

OpenRCA. Under the wrapper of Section 3.1, the three methods order as BARO (0.127), RCD (0.100), ϵ-Diagnosis (0.041). BARO's lead over RCD is +0.027 [0.002, 0.051], but Telecom reverses it

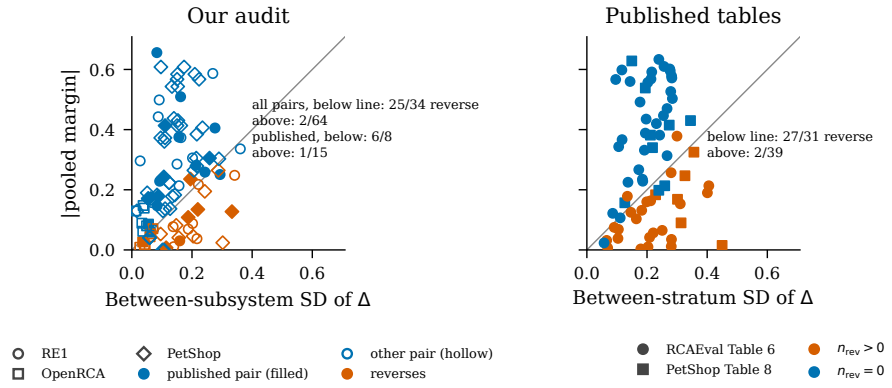

Fig. 3. Pooled margin $|\bar{\Delta}|$ against between-subsystem spread (sample standard deviation of Δ_s); the diagonal is margin = spread. Left: every pair of the main analysis (filled: published methods; orange: reverses on some subsystem). Right: the pairs inside two tables the benchmark papers publish, RCAEval's Table 6 (RE2-TT by fault type) and PetShop's Table 8 (by scenario and fault type), with the stratum in place of the subsystem.

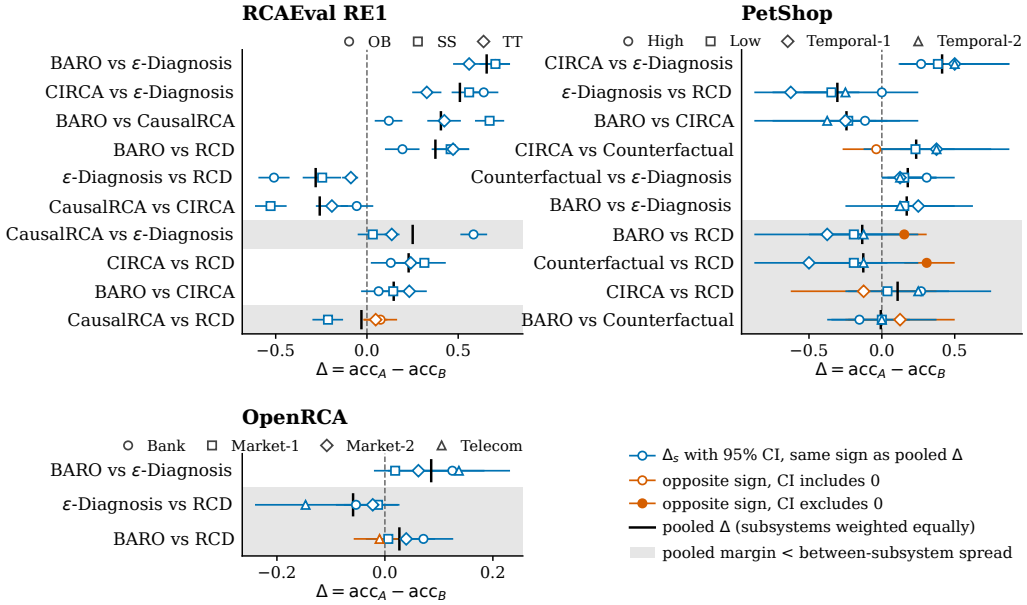

Fig. 4. Comparisons between published methods, by family. Each row is a pair A vs B; markers are subsystem differences Δ_s with 95% bootstrap CIs, the vertical tick the pooled difference (subsystems weighted equally). Orange: opposite sign to the pooled difference (filled: CI excludes 0). Grey rows: pooled margin below the between-subsystem spread, the pairs below the diagonal of Fig. 3.

(−0.010 [−0.058, 0.035]); the reversal rests on a few queries, and 2 of the three RCD seeds show a reversal, on different subsystems.

Table 2. Same application, two releases: subsystem differences Δ_s (A minus B) between published methods on RE1 and RE2. Bold: the two releases' 95% CIs lie on opposite sides of zero.

	Online Boutique		Sock Shop		Train Ticket	
Pair A vs B	RE1	RE2	RE1	RE2	RE1	RE2
BARO vs CausalRCA	+0.120	−0.063	**+0.672**	**−0.081**	+0.424	+0.459
BARO vs CIRCA	+0.064	−0.522	**+0.144**	**−0.711**	+0.232	+0.122
BARO vs ϵ-Diagnosis	+0.704	+0.111	+0.704	−0.089	+0.560	+0.667
CausalRCA vs CIRCA	−0.056	−0.459	−0.528	−0.630	−0.192	−0.337
CausalRCA vs ϵ-Diagnosis	+0.584	+0.174	+0.032	−0.007	+0.136	+0.207
CIRCA vs ϵ-Diagnosis	+0.640	+0.633	+0.560	+0.622	+0.328	+0.544

PetShop. The published methods are least stable here: 5 of 10 pairs reverse, 2 with CI support. Both CI-supported reversals come from one cell: RCD scores 0 of 26 on the high-traffic scenario, so BARO (+0.154 [0.038, 0.308]) and counterfactual attribution (+0.308 [0.154, 0.500]) beat it there while trailing it on pooled accuracy, with pooled CIs that include zero. The two temporal scenarios have 8 cases each; keeping only the two scenarios with at least 25 cases leaves 3 reversing pairs, 1 with CI support.

4.2 Same Applications, Second Release

RE2 is a second, independently collected release of RE1's three applications, with new fault instances, a sixth fault type, and added logs and traces. Its metric tables are also sampled differently (1,441 rows per case against 721 on RE1-SS) and record more metric kinds per service (70–78 columns on OB against 51–60 on RE1). The four published methods that use metrics only (BARO, CIRCA, ϵ-Diagnosis, CausalRCA) run with the same implementations on both releases. Of the 18 application-level comparisons available in both releases (Table 2), 12 keep their sign and 6 change it. 5 of the 6 involve BARO; the sixth, CausalRCA vs ϵ-Diagnosis on Sock Shop, moves from +0.032 [−0.051, 0.117] to −0.007 [−0.104, 0.089]. 2 comparisons have CIs on opposite sides of zero, both on Sock Shop: BARO vs CIRCA goes from +0.144 [0.064, 0.232] to −0.711 [−0.800, −0.611], and BARO vs CausalRCA from +0.672 to −0.081 [−0.163, −0.004]. Restricting RE2 to RE1's five fault types leaves the count at 12 of 18. Within RE2 itself, 10 of 28 published pairs reverse across the three applications (6 with CI support).

BARO's accuracy falls from 0.736 to 0.144 on OB and from 0.848 to 0.067 on SS. Its ranking is not the input column order, every case has a valid output, and the root cause is still among its first three candidates in 0.878 of the OB cases and 0.956 of the SS cases; what changes is the first position. RE2's reduced table adds disk I/O and socket metrics for every service, which RE1's tables do not contain, and BARO ranks the disk I/O of the application's database first in 77 of the 90 OB cases and 81 of the 90 SS cases, whatever the injected fault. Fang et al. [8] report top-1 accuracies of 0.14 on both applications for their own BARO baseline on RE2. What changed between the releases for this method is the set of metric kinds recorded, a part of the score's provenance that Section 5.6 takes up.

4.3 Choosing a Method for an Unseen Subsystem

A practitioner choosing a method for a new system has only the other subsystems to go by. Holding out each of the 11 subsystems in turn and ranking the published methods of its family on the remaining ones gives 36 adjacent-pair decisions (RE1 12, OpenRCA 8, PetShop 16). In 8 of them the higher-ranked method is less accurate on the held-out subsystem (2 with a CI excluding zero),

Table 3. Pooled winner (subsystems weighted equally) and its regret: accuracy of the best method on a subsystem minus the winner's accuracy there. All subsystems: winner chosen on all subsystems of the family. Held out: for each subsystem, winner chosen on the other subsystems only (not computed for RE2, which we use only across releases).

				All subsystems		Held out	
Family	Layer	Pooled winner	Winner pooled acc.	Regret > 0	Max regret (subsystem)	Regret > 0	Max
RCAEval RE1	Published	BARO	0.715	0 of 3	0	0 of 3	0
RCAEval RE1	All	BARO	0.715	1 of 3	0.080 (RE1-SS)	2 of 3	0.280
RCAEval RE2	Published	CIRCA	0.663	1 of 3	0.256 (RE2-TT)	–	–
RCAEval RE2	All	CD-1min	0.822	2 of 3	0.044 (RE2-OB)	–	–
OpenRCA	Published	BARO	0.127	1 of 4	0.010 (Telecom)	1 of 4	0.010
OpenRCA	All	Alert-count	0.189	2 of 4	0.017 (Market-1)	4 of 4	0.078
PetShop	Published	CIRCA	0.445	2 of 4	0.125 (Temporal-1)	2 of 4	0.125
PetShop	All	Ranked correlation	0.639	1 of 4	0.375 (Temporal-2)	3 of 4	0.375

and 2 further decisions are ties. The 8 wrong choices fall on pairs that Section 4.1 places below the line. On RE1 all 3 are RCD vs CausalRCA, the pair whose pooled sign changes with the seed; on OpenRCA the one is BARO vs RCD on Telecom, the reversal with a CI including zero; on PetShop 2 are on the high-traffic scenario, among them RCD vs BARO, and 2 on the 8-case temporal scenario. None of the 11 decisions that involve ϵ-Diagnosis is wrong. Computed on the remaining subsystems alone, margin and spread do not single out the wrong choices: 4 of the 19 decisions whose margin exceeds the spread there are wrong, and 4 of the 17 others (replication package). Table 3 gives the cost of such a choice as regret. The pooled winner chosen without the held-out subsystem has positive regret on 3 of the 11 subsystems (up to 0.125), and with probes and shipped baselines on 9 of 11 (up to 0.375). The largest costs are concrete: on RE2, CIRCA has the highest pooled accuracy (0.663) but scores 0.544 on Train Ticket against SimpleRCA's 0.800; on PetShop the pooled winner CIRCA trails another published method on the high-traffic scenario by 0.038.

4.4 Probes and Shipped Baselines

Adding the three probes and PetShop's shipped baselines gives 98 pairs in the main families, of which 27 reverse and 8 with CI support (RE1 8/4 of 28; OpenRCA 7/0 of 15; PetShop 12/4 of 55). The probes do not win everywhere or lose everywhere: CD-1min is the best method on RE1-SS but ranks below BARO on RE1-TT, and alert-count leads OpenRCA's pooled ranking. On PetShop, the shipped ranked-correlation baseline has the highest pooled accuracy (0.639), and its pooled advantage over every published method except CIRCA has a CI that excludes zero. A leaderboard that lists only published methods therefore ranks methods that a shipped baseline of the same benchmark outperforms.

4.5 Across Benchmark Families

6 methods (BARO, RCD, ϵ-Diagnosis, and the three probes) have outputs on all 11 subsystems, so their comparisons can be followed across families by the signs of their subsystem differences (Section 3.4; per-pair tables in the replication package). Of the 15 pairs, 10 have subsystem differences of both signs; the other 5 all involve ϵ-Diagnosis, which is never more accurate than any of the other five on any subsystem, so a ranking that places it last holds everywhere. Dropping any one family leaves both signs in 28 of 30 pair-by-family cases, and dropping any two subsystems in 549

Table 4. Pipeline stages that change comparison conclusions. Comp.: comparisons (one method pair on one subsystem); Sign: comparisons whose Δ_s changes sign; Rev.: comparisons that are a reversal under exactly one of the two conditions. All pairs include probes and shipped baselines; –: no published pair has outputs under both conditions. Seed: the three-seed mean of a randomized method against any single seed, over the comparisons that involve such a method (Section 5.3).

		Published methods			All pairs		
Factor	Family	Comp.	Sign	Rev.	Comp.	Sign	Rev.
Input representation	RCAEval RE1	12	5	1	42	16	8
	RCAEval RE2	–	–	–	18	5	7
Silent failure (duplicated header)	RCAEval RE1	30	2	2	84	3	4
Denominator (valid outputs only)	RCAEval RE1	30	0	0	84	0	0
	RCAEval RE2	71	1	1	146	2	2
	OpenRCA	12	0	0	60	0	0
	PetShop	40	0	0	220	0	0
Fault re-weighting	RCAEval RE1	30	0	0	84	0	0
	RCAEval RE2	71	0	0	146	0	0
	OpenRCA	12	3	3	60	13	15
	PetShop	40	1	1	220	2	1
Scoring (strict = 1)	OpenRCA	12	4	0	60	11	8
Scoring (≥ 0.5)	OpenRCA	12	0	0	60	3	3
Seed (one seed vs. three-seed mean)	RCAEval RE1	21	3	4	39	5	6
	RCAEval RE2	19	2	2	28	4	4
	OpenRCA	8	3	3	20	3	3

of 550: no order of the other five methods, on whatever scale the families were averaged, holds on every subsystem.

5 RQ2 (Provenance): How the Score Was Produced

A benchmark score is the output of a pipeline, and the released pipeline leaves choices open at four stages: which representation of the telemetry the method reads, what happens when the method fails, how invalid outputs enter the denominator, and how cases are weighted and scored. RQ1 read each subsystem difference as a property of the method and the subsystem; this section asks which choices of the released pipeline produced it. The duplicated header of Section 3.3 is one such choice's effect, a subsystem anomaly that disappears once the input is fixed. For each stage we compare the reference configuration of Section 3.3 with one alternative on the same cases and the same methods, and record for every pair and subsystem whether Δ_s changes sign and whether the comparison's reversal status changes (Table 4). The stages are not equal: the first two produced opposite conclusions with CIs excluding zero on both sides; the last two moved, on OpenRCA, only differences that were already close to zero.

5.1 Input Representation

RE1-SS and RE1-TT can be read from the raw export or from the reduced table (Section 3.3), and the two readings do not rank the methods the same way. Figure 5 shows each method's accuracy under both. BARO falls from 0.848 to 0.200 on SS and from 0.560 to 0.120 on TT, and CIRCA from 0.704 to 0.200 and from 0.328 to 0.000, while alert-count and CD-1min improve on TT and ϵ-Diagnosis and RCD barely move. The raw export is not a uniform penalty; it reorders the methods. The release

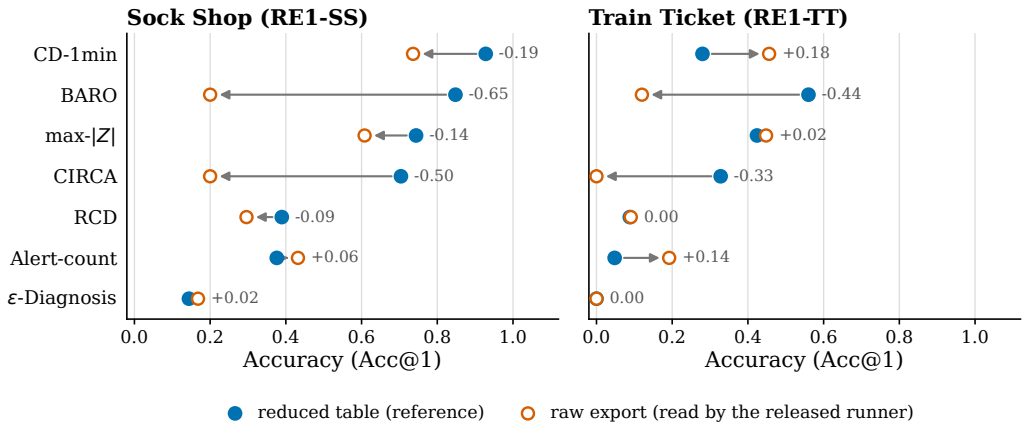

Fig. 5. RE1 accuracy of each method on the reduced table (the reference) and on the raw export that the released runner reads.

does not state which table its reported numbers use, and a reader who runs it as shipped obtains the raw numbers.

Among published methods, 5 of the 12 SS and TT comparisons change sign, the largest share at any stage. CIRCA leads RCD by +0.315 [0.195, 0.432] on SS and +0.240 [0.149, 0.328] on TT with the reduced tables; with the raw exports RCD leads, by −0.096 [−0.184, −0.005] and −0.091 [−0.139, −0.048]. All four intervals exclude zero: two analyses of the same released cases, each a literal reading of the release, report opposite conclusions with the same apparent support. On SS the best published method changes from BARO to RCD. The raw exports add 2 reversals among published pairs (1 with CI support) that are absent with the reduced tables; 1 of them is on OB, whose data are the same under both conditions, because the pooled sign of CIRCA vs RCD changes (Table 4 counts the SS and TT comparisons). Among all pairs 16 of 42 comparisons change sign, each condition showing reversals the other lacks: the input representation replaces one set of conclusions with another. RE2 ships the same two kinds of table, and there the runner reads the reduced one; the raw table changes 5 of 18 comparisons among the methods we could run on it.

5.2 Silent Failures

A method that fails inside the pipeline but returns a complete ranking leaves no trace on the leaderboard; the ranking is scored like any other. Both failures found in Section 3.3 are of this kind. Under the drifted dependency, CausalRCA returns the input column order on every case we tested, so a comparison that involves it on the release as shipped compares the column order of the input file, not the method. The duplicated header in 50 RE1-OB tables has an effect we can measure, because CIRCA and ϵ-Diagnosis run on the same cases before and after the fix. On the files as shipped, CIRCA's accuracy on RE1-OB is 0.432; after dropping the duplicated column it is 0.672 (0.220 and 0.820 on the 50 affected cases). As shipped, CausalRCA beats CIRCA on OB by +0.184 [0.077, 0.291], a CI-supported reversal of CIRCA's pooled lead; after the fix the difference is −0.056 [−0.144, 0.035], and CIRCA vs RCD on OB changes from −0.115 to +0.131. Of 30 comparisons between published methods on RE1, 2 change sign, and the shipped files produce 2 reversals (1 with CI support) that disappear after the fix. Neither failure raises an error or lowers coverage; the one check that reveals both is whether a method's ranking equals the order of the input columns.

5.3 Denominator and Reported Numbers

25 of 149 method-by-subsystem cells (seeds counted separately) have coverage below 1, for method-specific reasons: TraceRCA and MicroRank fail on the same 17 RE2-TT cases with a missing-key error, leaving 73 valid outputs of 90; ϵ-Diagnosis returns three metrics per query, none of which belongs to the candidate set in most OpenRCA Telecom queries (19 of 51 valid); and one seed of CausalRCA learns an empty graph in 43 RE2-TT cases (47 valid). Scoring valid outputs only, instead of all cases, changes 1 of 71 comparisons between published methods on RE2 and 2 of 146 among all pairs, all on RE2-TT (CIRCA vs TraceRCA moves from +0.011 to −0.113), and none of the 364 comparisons in the other families. Two things a reported number does not state matter when it is compared with a published one: the denominator and the seed. Our TraceRCA and MicroRank outputs match every per-fault value that RCAEval reports for RE2-TT (18 cells each, within 0.01) only if the 17 failed cases are dropped, although the paper's definition of the metric divides by all cases. Over all cases, TraceRCA's Acc@1 is 0.533 instead of the reported 0.66. CausalRCA's code defines a seed but never applies it, and on RE2-TT, where RCAEval reports 0.22, our three seeds give 0.267, 0.133, and 0.222. Taking any single seed instead of the three-seed mean changes the sign of 3 of the 21 comparisons between published methods that involve a randomized method on RE1, 2 of 19 on RE2, and 3 of 8 on OpenRCA (last row of Table 4).

5.4 Fault Composition

A pooled score also mixes fault types, which are not equally frequent in every subsystem. We re-weight each subsystem so that the fault classes it shares with the rest of its family count equally. Both RCAEval releases contain the same number of cases for every fault type in every application, so no comparison there changes. OpenRCA's four subsystems share only CPU and network faults; restricting each subsystem to these and weighting them equally changes the sign of 3 of 12 comparisons between published methods and 13 of 60 overall, none with a CI that excludes zero; on PetShop (latency and availability faults), 1 of 40 and 2 of 220 change.

5.5 Scoring Rule

OpenRCA's official score gives partial credit. Counting only fully correct answers changes the sign of 4 of 12 comparisons between published methods (11 of 60 overall); requiring a score of at least 0.5 changes 0 and 3. All 4 sign changes between published methods under the strict rule are differences that become exactly zero, and none changes a reversal status (Table 4): on Market-1, BARO, RCD, and ϵ-Diagnosis score 0.000, 0.000, and 0.000 under the strict rule and 0.057, 0.051, and 0.038 with partial credit. Under the strict rule the published methods are indistinguishable on that subsystem; the ordering reported under partial credit comes entirely from partially correct answers. Among all pairs, 7 of the reversal-status changes remain non-zero differences and 1 is a reversal that becomes a tie. Every method's score on OpenRCA lies between 0.036 and 0.242 under the official rule and between 0.000 and 0.157 under the strict one, so the differences a scoring rule can move there are of this size. Where scores are not at the floor the rule moves more: on RE2-OB, BARO's accuracy is 0.144 at top-1 and 0.878 at top-3 (Section 4.2).

5.6 Reading the Four Stages Together

The first two stages changed conclusions between published methods with CI support; the last two changed, on OpenRCA, the sign only of differences that were close to zero in the reference analysis, or turned them into ties. Both silent failures were found in RCAEval, the one family whose released runner we could execute as shipped; OpenRCA releases no runner for these methods, and we found none in PetShop's harness. The largest provenance effect we measured is a choice of the release

rather than of the runner: which metric kinds it records (Section 4.2) changed BARO's accuracy by as much as the input representation did. The denominator, the fault weighting, and the scoring rule are choices every benchmark makes, and they are the ones a reader can usually see. An audit should therefore start with what is least visible, which table the runner reads, which metric kinds it contains, and whether rankings equal the input column order, then ask how invalid outputs enter the denominator, and only then turn to the weighting and the scoring rule.

6 Can Either Premise Be Checked From Published Papers?

The two premises can be checked from a paper only if it reports the per-system scores behind its pooled comparisons, releases enough to attach an uncertainty to them, and states how its scores were produced. We read 23 microservice RCA method and benchmark papers from 2021 to 2026, a convenience sample built from the benchmark papers, the method papers they compare, and the papers citing them for which we obtained the full text (Table 5), and checked these three things. A paper counts as reporting per-system results if it gives a score for each evaluated system or dataset; a release counts as containing per-case outputs if a file in it gives a method's ranking or prediction for an identifiable case, and as complete if such files exist for every method on every dataset of the main comparison. Scope is reported: 17 of the 23 papers evaluate more than one system, and all 17 give a score per system, so a reader can see where a pooled lead does not hold, but not with what uncertainty. Checking the repositories and archives linked from each paper on 2 and 3 October 2026, we found 7 papers whose release contains per-case outputs of at least one method. For 3 of them (BARO [24], RCAEval [26], and a study that shares RCAEval's repository [25]) the outputs are those a tutorial notebook saved for a single case. A further 3 release the outputs of the authors' own method on their test cases: one archived run of OpenRCA's agent [38], the rankings of CausalRCA [37], and the predictions of CHASE [45]. One, CIRCA [17], releases the rankings of every method of its simulation study, but not of its production dataset. For 2 papers a linked release had expired or could not be reached. None of the 21 papers whose releases we could check releases per-case outputs for every method on every dataset of its main comparison, so for those comparisons a reader cannot compute a paired difference, its uncertainty, or a coverage figure.

Whether a number was computed on the raw or the reduced table is a question that even the benchmark's own paper does not answer. RCAEval's paper describes the file format of its release but not which of the two tables its reported numbers use, and Section 5.1 shows that the two rank the methods differently. Superiority claims are nonetheless stated over the benchmark as a whole, for example "Extensive experiments conducted on the synthetic and real-world microservice-based datasets demonstrate that RUN noticeably outperforms the state-of-the-art root cause analysis methods" [20].

7 Discussion and Recommendations

7.1 Stating Both Premises

A per-subsystem table states the first premise, but pooled numbers will continue to be quoted, so we suggest that every pooled comparison carry a line of context with one part per premise:

scope	pooled Δ [CI] · subsystems won/tied/lost of k · worst subsystem Δ [CI]
provenance	coverage · input table · invalid outputs

For RCD vs BARO on PetShop this line reads: +0.135 [−0.041, 0.300] · 3/0/1 of 4 · high traffic −0.154 [−0.308, −0.038] · 1.00 vs 1.00 · official harness · scored 0. A reader sees at once that the pooled CI includes zero, that the lead holds on three of four scenarios, that the one loss has a CI excluding zero, and on which input and denominator the numbers rest; the pooled number alone, +0.135, says none of this. The subsystem count and the worst subsystem carry the spread that

Table 5. The 23 surveyed papers. Per-system: a score per evaluated system is reported (single system: one system evaluated). Per-case outputs released: what the release linked from the paper contains, checked on 2 and 3 October 2026. All methods: whether per-case outputs exist for every method on every dataset of the main comparison (partial: for every method of one experiment).

Paper	Year	Venue	Per-system	Per-case outputs released	All methods
BARO [24]	2024	FSE 2024	yes	one demonstration case	no
CIRCA [17]	2022	KDD 2022	yes	all methods, simulation study only	partial
RCD [11]	2022	NeurIPS 2022	yes	–	no
TraceRCA [18]	2021	IWQoS 2021	yes	release unreachable	–
RCAEval [26]	2025	WWW 2025 Companion	single system	one demonstration case	no
SimpleRCA [8]	2026	FSE 2026	yes	–	no
OpenRCA [38]	2025	ICLR 2025	yes	own method, all test cases	no
PetShop [9]	2024	CLeaR 2024	single system	–	no
Eadro [16]	2023	ICSE 2023	yes	–	no
mABC [44]	2024	EMNLP 2024 Findings	yes	–	no
MULAN [47]	2024	WWW 2024	yes	–	no
OCEAN [46]	2025	IEEE BigData 2025	yes	–	no
TVDiag [36]	2025	TOSEM	yes	–	no
CHASE [45]	2024	arXiv 2024	yes	own method, all test cases	no
How Far Are We [25]	2024	ASE 2024	yes	one demonstration case	no
LEMMA-RCA [48]	2024	arXiv 2024	yes	–	no
SparseRCA [39]	2024	ISSRE 2024	single system	–	no
TraceContrast [41]	2024	ICSE 2024	single system	–	no
TraceDiag [6]	2023	ESEC/FSE 2023	single system	–	no
RUN [20]	2024	AAAI 2024	yes	–	no
DynaCausal [42]	2025	arXiv 2025	yes	–	no
CausalRCA [37]	2023	JSS 2023	single system	own method, all test cases	no
Too Many Cooks [43]	2025	ISSRE 2025	yes	release unreachable	–

Section 4.1 found to decide whether a lead holds everywhere. The line is a reporting format, not a decision rule, and every quantity in it can be computed from per-case outputs. Where the lead does not hold everywhere, a count such as “leads on 3 of 4 subsystems” says more than “outperforms”.

7.2 For Benchmark Maintainers

Each recommendation below corresponds to a problem we verified.

(i) The runner should name the input table and preprocessing it uses, in code and not only in documentation; the two readings of RE1 reversed a comparison between published methods with CI support on both sides (Section 5.1).

(ii) Dependency versions should be pinned, and a method that fails inside the pipeline should fail visibly rather than return a default ranking; under an unpinned dependency one method returned its input column order on every case (Section 5.2).

(iii) Input tables should be validated before release; a duplicated header in 50 tables produced a CI-supported reversal that disappeared once removed.

(iv) Coverage and a full-case denominator should accompany every score; reported values that drop failed cases cannot be matched by a reader who keeps them (Section 5.3).

(v) Per-case outputs of every reported method should be released, so that readers can compute paired differences and intervals, which none of the papers we could check allows (Section 6).

The current upstream release of RCAEval includes fixes for (ii) and (iii).

7.3 Relation to Benchmark-Difficulty Critiques

Fang et al. [8] show that a simple rule-based method reaches the reported state of the art on existing RCA benchmarks and propose a harder benchmark. We hold the benchmarks fixed and ask which of the comparisons drawn from them hold across their subsystems and which part of a score the released pipeline produced. The two concerns are independent and their remedies combine: a

harder benchmark still needs its comparisons scoped and their provenance stated, and both stated on an easy benchmark still overstate what methods can do. Where the results overlap they agree: the authors' SimpleRCA code is also best on RE2-TT in our runs.

8 Related Work

Benchmark validity in machine learning. Pooled leaderboards have been criticized for hiding condition-specific behavior and for depending on which benchmarks are included [1, 5, 19]. Test-set replication and label errors can reorder methods [22, 28], and hidden stratification and underspecification describe models whose average performance hides failures on subgroups [4, 23, 31], and checklists for benchmark quality ask for documentation and reproducible releases [27, 29]. We apply this view to RCA benchmarks, where subsystems are identifiable applications, per-case scores are available, and the release itself can be under-specified.

Empirical software engineering. Guidelines for empirical research in software engineering ask that a study state its population, treatments, and analysis in enough detail to be repeated [15], and catalogues of recurring problems in software analytics papers include inadequate reporting of data preparation and conclusions whose stability is never tested [21]. In defect prediction, the research group explained more of the variation in reported performance than the classifier did [33]; cross-project experiments found that whether a predictor transfers depends on the projects involved [50], a benchmark of cross-project approaches found their ranking to depend on the datasets and metrics used [10], and the model-validation technique was shown to change performance estimates and rankings [34]. We ask the same questions of RCA benchmarks: whether a comparison holds on each evaluated system, and which part of a score the evaluation procedure produced.

RCA benchmarks and methods. Offline RCA benchmarks include RCAEval [26], OpenRCA [38], PetShop [9], and LEMMA-RCA [48]; AIOpsLab and ITBench evaluate agents against live systems, and Cloud-OpsBench against recorded state snapshots replayed through diagnostic tools [3, 13, 35]. Audited methods span change-point detection (BARO [24]), causal inference and discovery (CIRCA [17], RCD [11], CausalRCA [37], counterfactual attribution [2]), anomaly scoring (ϵ-Diagnosis [32]), and trace analysis (TraceRCA [18], MicroRank [40]).

Evaluating RCA evaluation. Pham et al. [25] compare causal-inference RCA methods on common data and find that no method stands out in all situations. We do not re-establish that winners differ; we ask, for the pairwise comparisons a released benchmark supports, on which subsystems a pooled lead holds and which part of it the evaluation pipeline produced, with matched per-case outputs that others can recheck. Fang et al. [8] question benchmark difficulty (Section 7.3). Studies of LLM agents for cloud RCA analyze why agents fail [14, 30]; they concern a different evaluation mode from ours.

9 Threats to Validity

Construct validity. RCAEval and PetShop score top-1 hits, while OpenRCA gives partial credit for component, reason, and time. We compare differences within a family, except for the cross-family set (Section 4.5), where we compare only the signs of subsystem differences and do not average scores across families. A reversal can be defined by the point estimate or by the CI; we report both, and the conclusions of Section 4 hold under either.

Internal validity. Our results depend on running released code correctly, which is exactly where we found problems. Our own first RE1 runs also read the raw exports; we noticed only because BARO's Avg@5 did not match its authors' numbers. We guard against similar errors in four ways. We anchor implementations to published numbers where possible (BARO on RE1, SimpleRCA on RE2 and RE3, and TraceRCA and MicroRank on RE2-TT when invalid outputs are dropped); we check every ranking against the input column order; we apply the fixes of the current upstream

release; and we record the configuration of every run. The OpenRCA wrapper that turns metric rankings into answers is our design; it is the same for every method, and queries where it falls back to a default answer score 0. Because methods favour different kinds of metrics, the mapping from metric type to failure reason may suit some methods better than others, so our OpenRCA results hold under this wrapper and not necessarily under another. The three probes have no parameter chosen on the data they are scored on: max-$|Z|$ and alert-count are fixed z-score rules, and CD-1min's thresholds were set on OpenRCA before this audit, so its OpenRCA results are not out of sample while its results on the other families are. Two randomized methods are averaged over three seeds; the last row of Table 4 gives what a single seed changes, and the per-seed values are in the replication package.

External validity. We audit three benchmark families with $k = 11$ subsystems in the main analysis; they are not a sample from a population of deployments, and we make no claim beyond them. RE2 uses the same applications as RE1, so it tests carry-over between releases, not between systems; it differs from RE1 in collection, sampling, and recorded metric kinds as well as in faults, and the released data do not allow these to be separated. Two PetShop scenarios have only 8 cases, and we report results without them. Our conclusions are limited to offline benchmarks with matched cases; interactive agent benchmarks may behave differently.

Conclusion validity. We examine many comparisons, and subsystem CIs are not adjusted for multiplicity; a few CI-supported reversals are expected by chance, which is why we report counts and do not interpret single reversals. At a Bonferroni-adjusted level over the 82 subsystem comparisons between published methods in the main families, 31 of the 46 subsystem CIs that exclude zero still do so, and 1 of the 2 CI-supported reversals of RQ1 remains (counterfactual attribution vs RCD on the high-traffic scenario). The reversals of RQ2 have CIs excluding zero on both sides under both conditions. The survey of Section 6 is a convenience sample of 23 papers checked by one author on two dates through the links given in the papers; releases can change afterwards, and for 2 papers a linked release could not be reached.

10 Conclusion

A pooled leaderboard reports one number per method and leaves unstated on which systems the advantage holds and how the score was produced. In three RCA benchmark families, whether a pooled lead between published methods held on every subsystem was decided by a quantity the leaderboard withholds, the spread of the lead across subsystems: leads whose margin exceeded it held everywhere, most leads below it reversed, and every wrong choice on a held-out subsystem was of the second kind. On a second release of the same applications, one method's response to newly recorded metric kinds accounted for five of the six comparisons that changed sign, and the input table, the handling of failed outputs, and the denominator of the released pipelines changed conclusions without raising an error. Stating the subsystems and the spread next to every pooled comparison, naming the input table and the denominator, and releasing the per-case outputs behind it would let readers check what the leaderboard does not say.

Data Availability

The replication package is available at https://anonymous.4open.science/r/pooled-leaderboard-audit-replication-8DB8/. It contains the frozen per-case table with its SHA-256 digest, the run ledger, the run and analysis scripts, the adapted SimpleRCA code, the integrity-check diagnostics, the survey data, and additional result tables; a quick check regenerates every number, table, and figure of the paper from the per-case table in a few minutes, and a full rerun starts from the public benchmark data, which the package does not redistribute.

References

[1] Samuel R. Bowman and George E. Dahl. 2021. What Will it Take to Fix Benchmarking in Natural Language Understanding?. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*. Association for Computational Linguistics, 4843–4855.

[2] Kailash Budhathoki, Lenon Minorics, Patrick Blöbaum, and Dominik Janzing. 2022. Causal Structure-Based Root Cause Analysis of Outliers. In *Proceedings of the 39th International Conference on Machine Learning (ICML 2022) (Proceedings of Machine Learning Research, Vol. 162)*. PMLR, 2357–2369.

[3] Yinfang Chen, Manish Shetty, Gagan Somashekar, Minghua Ma, Yogesh Simmhan, Jonathan Mace, Chetan Bansal, Rujia Wang, and Saravan Rajmohan. 2025. AIOpsLab: A Holistic Framework to Evaluate AI Agents for Enabling Autonomous Clouds. In *Proceedings of the Eighth Conference on Machine Learning and Systems (MLSys 2025)*.

[4] Alexander D'Amour, Katherine A. Heller, Dan Moldovan, Ben Adlam, Babak Alipanahi, Alex Beutel, Christina Chen, Jonathan Deaton, Jacob Eisenstein, Matthew D. Hoffman, Farhad Hormozdiari, Neil Houlsby, Shaobo Hou, Ghassen Jerfel, Alan Karthikesalingam, Mario Lucic, Yian Ma, Cory McLean, Diana Mincu, Akinori Mitani, Andrea Montanari, Zachary Nado, Vivek Natarajan, Christopher Nielson, Thomas F. Osborne, Rajiv Raman, Kim Ramasamy, Rory Sayres, Jessica Schrouff, Martin Seneviratne, Shannon Sequeira, Harini Suresh, Victor Veitch, Max Vladymyrov, Xuezhi Wang, Kellie Webster, Steve Yadlowsky, Taedong Yun, Xiaohua Zhai, and D. Sculley. 2022. Underspecification Presents Challenges for Credibility in Modern Machine Learning. *Journal of Machine Learning Research* 23, 226 (2022), 1–61.

[5] Mostafa Dehghani, Yi Tay, Alexey A. Gritsenko, Zhe Zhao, Neil Houlsby, Fernando Diaz, Donald Metzler, and Oriol Vinyals. 2021. The Benchmark Lottery. *arXiv preprint arXiv:2107.07002* (2021).

[6] Ruomeng Ding, Chaoyun Zhang, Lu Wang, Yong Xu, Minghua Ma, Xiaomin Wu, Meng Zhang, Qingjun Chen, Xin Gao, Xuedong Gao, Hao Fan, Saravan Rajmohan, Qingwei Lin, and Dongmei Zhang. 2023. TraceDiag: Adaptive, Interpretable, and Efficient Root Cause Analysis on Large-Scale Microservice Systems. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. 1762–1773.

[7] Bradley Efron and Robert J. Tibshirani. 1993. *An Introduction to the Bootstrap*. Monographs on Statistics and Applied Probability, Vol. 57. Chapman and Hall, New York.

[8] Aoyang Fang, Songhan Zhang, Yifan Yang, Haotong Wu, Junjielong Xu, Xuyang Wang, Rui Wang, Manyi Wang, Qisheng Lu, and Pinjia He. 2026. Rethinking the Evaluation of Microservice RCA with a Fault Propagation-Aware Benchmark. *Proceedings of the ACM on Software Engineering* 3, FSE (2026), 1605–1627.

[9] Michaela Hardt, William R. Orchard, Patrick Blöbaum, Elke Kirschbaum, and Shiva Kasiviswanathan. 2024. The PetShop Dataset — Finding Causes of Performance Issues across Microservices. In *Proceedings of the Third Conference on Causal Learning and Reasoning (Proceedings of Machine Learning Research, Vol. 236)*. PMLR, 957–978.

[10] Steffen Herbold, Alexander Trautsch, and Jens Grabowski. 2018. A Comparative Study to Benchmark Cross-Project Defect Prediction Approaches. *IEEE Transactions on Software Engineering* 44, 9 (2018), 811–833.

[11] Azam Ikram, Sarthak Chakraborty, Subrata Mitra, Shiv Kumar Saini, Saurabh Bagchi, and Murat Kocaoglu. 2022. Root Cause Analysis of Failures in Microservices through Causal Discovery. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*. 31158–31170.

[12] Joanna IntHout, John P. A. Ioannidis, Maroeska M. Rovers, and Jelle J. Goeman. 2016. Plea for Routinely Presenting Prediction Intervals in Meta-Analysis. *BMJ Open* 6, 7 (2016), e010247.

[13] Saurabh Jha, Rohan Arora, Yuji Watanabe, Takumi Yanagawa, Yinfang Chen, Jackson Clark, Bhavya Bhavya, Mudit Verma, et al. 2025. ITBench: Evaluating AI Agents across Diverse Real-World IT Automation Tasks. *arXiv preprint arXiv:2502.05352* (2025).

[14] Taeyoon Kim, Woohyeok Park, Hoyeong Yun, and Kyungyong Lee. 2026. Why Do AI Agents Systematically Fail at Cloud Root Cause Analysis? *arXiv preprint arXiv:2602.09937* (2026).

[15] Barbara A. Kitchenham, Shari Lawrence Pfleeger, Lesley M. Pickard, Peter W. Jones, David C. Hoaglin, Khaled El Emam, and Jarrett Rosenberg. 2002. Preliminary Guidelines for Empirical Research in Software Engineering. *IEEE Transactions on Software Engineering* 28, 8 (2002), 721–734.

[16] Cheryl Lee, Tianyi Yang, Zhuangbin Chen, Yuxin Su, and Michael R. Lyu. 2023. Eadro: An End-to-End Troubleshooting Framework for Microservices on Multi-source Data. In *Proceedings of the 45th IEEE/ACM International Conference on Software Engineering (ICSE)*. 1750–1762.

[17] Mingjie Li, Zeyan Li, Kanglin Yin, Xiaohui Nie, Wenchi Zhang, Kaixin Sui, and Dan Pei. 2022. Causal Inference-Based Root Cause Analysis for Online Service Systems with Intervention Recognition. In *KDD '22: The 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Washington, DC, USA, August 14–18, 2022*, Aidong Zhang and Huzefa Rangwala (Eds.). ACM, 3230–3240.

[18] Zeyan Li, Junjie Chen, Rui Jiao, Nengwen Zhao, Zhijun Wang, Shuwei Zhang, Yanjun Wu, Long Jiang, Leiqin Yan, Zikai Wang, Zhekang Chen, Wenchi Zhang, Xiaohui Nie, Kaixin Sui, and Dan Pei. 2021. Practical Root Cause Localization for Microservice Systems via Trace Analysis. In *2021 IEEE/ACM 29th International Symposium on Quality of Service*

(IWQoS). IEEE, 1–10.

[19] Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, Deepak Narayanan, et al. 2023. Holistic Evaluation of Language Models. *Transactions on Machine Learning Research (TMLR)* (2023).

[20] Cheng-Ming Lin, Ching Chang, Wei-Yao Wang, Kuang-Da Wang, and Wen-Chih Peng. 2024. Root Cause Analysis in Microservice Using Neural Granger Causal Discovery. In *Proceedings of the 38th AAAI Conference on Artificial Intelligence (AAAI 2024)*. 206–213.

[21] Tim Menzies and Martin Shepperd. 2019. "Bad smells" in software analytics papers. *Information and Software Technology* 112 (2019), 35–47.

[22] Curtis G. Northcutt, Anish Athalye, and Jonas Mueller. 2021. Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks. In *Proceedings of the 35th Conference on Neural Information Processing Systems (NeurIPS 2021) Datasets and Benchmarks Track*.

[23] Luke Oakden-Rayner, Jared Dunnmon, Gustavo Carneiro, and Christopher Ré. 2020. Hidden Stratification Causes Clinically Meaningful Failures in Machine Learning for Medical Imaging. In *Proceedings of the ACM Conference on Health, Inference, and Learning*. 151–159.

[24] Luan Pham, Huong Ha, and Hongyu Zhang. 2024. BARO: Robust Root Cause Analysis for Microservices via Multivariate Bayesian Online Change Point Detection. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 2214–2237.

[25] Luan Pham, Huong Ha, and Hongyu Zhang. 2024. Root Cause Analysis for Microservice System based on Causal Inference: How Far Are We?. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. ACM, 706–715.

[26] Luan Pham, Hongyu Zhang, Huong Ha, Flora Salim, and Xiuzhen Zhang. 2025. RCAEval: A Benchmark for Root Cause Analysis of Microservice Systems with Telemetry Data. In *Companion Proceedings of the ACM on Web Conference 2025 (WWW Companion)*. ACM, 777–780.

[27] Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Larivière, Alina Beygelzimer, Florence d'Alché Buc, Emily Fox, and Hugo Larochelle. 2021. Improving Reproducibility in Machine Learning Research (A Report from the NeurIPS 2019 Reproducibility Program). *Journal of Machine Learning Research* 22, 164 (2021), 1–20.

[28] Benjamin Recht, Rebecca Roelofs, Ludwig Schmidt, and Vaishaal Shankar. 2019. Do ImageNet Classifiers Generalize to ImageNet?. In *Proceedings of the 36th International Conference on Machine Learning (ICML 2019)*. 5389–5400.

[29] Anka Reuel, Amelia Hardy, Chandler Smith, Max Lamparth, Malcolm Hardy, and Mykel J. Kochenderfer. 2024. BetterBench: Assessing AI Benchmarks, Uncovering Issues, and Establishing Best Practices. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024) Datasets and Benchmarks Track*.

[30] Evelien Riddell, James Riddell, Gengyi Sun, Michał Antkiewicz, and Krzysztof Czarnecki. 2026. Stalled, Biased, and Confused: Uncovering Reasoning Failures in LLMs for Cloud-Based Root Cause Analysis. In *Proceedings of the 2026 IEEE/ACM Third International Conference on AI Foundation Models and Software Engineering (FORGE)*. ACM, 172–183.

[31] Shiori Sagawa, Pang Wei Koh, Tatsunori B. Hashimoto, and Percy Liang. 2020. Distributionally Robust Neural Networks for Group Shifts: On the Importance of Regularization for Worst-Case Generalization. In *Proceedings of the Eighth International Conference on Learning Representations (ICLR 2020)*.

[32] Huasong Shan, Yuan Chen, Haifeng Liu, Yunpeng Zhang, Xiao Xiao, Xiaofeng He, Min Li, and Wei Ding. 2019. ϵ-Diagnosis: Unsupervised and Real-time Diagnosis of Small-window Long-tail Latency in Large-scale Microservice Platforms. In *Proceedings of the World Wide Web Conference (WWW 2019)*. 3215–3222.

[33] Martin Shepperd, David Bowes, and Tracy Hall. 2014. Researcher Bias: The Use of Machine Learning in Software Defect Prediction. *IEEE Transactions on Software Engineering* 40, 6 (2014), 603–616.

[34] Chakkrit Tantithamthavorn, Shane McIntosh, Ahmed E. Hassan, and Kenichi Matsumoto. 2017. An Empirical Comparison of Model Validation Techniques for Defect Prediction Models. *IEEE Transactions on Software Engineering* 43, 1 (2017), 1–18.

[35] Yilun Wang, Guangba Yu, Haiyu Huang, Yujie Huang, Zirui Wang, Pengfei Chen, and Michael R. Lyu. 2026. Cloud-OpsBench: A Reproducible Benchmark for Agentic Root Cause Analysis in Cloud Systems. *arXiv preprint arXiv:2603.00468* (2026).

[36] Shuaiyu Xie, Jian Wang, Hanbin He, Zhihao Wang, Yuqi Zhao, Neng Zhang, and Bing Li. 2025. TVDiag: A Task-oriented and View-invariant Failure Diagnosis Framework for Microservice-based Systems with Multimodal Data. *ACM Transactions on Software Engineering and Methodology* 35, 2 (2025).

[37] Ruyue Xin, Peng Chen, and Zhiming Zhao. 2023. CausalRCA: Causal inference based precise fine-grained root cause localization for microservice applications. *Journal of Systems and Software* 203 (2023), 111724.

[38] Junjielong Xu, Qinan Zhang, Zhiqing Zhong, Shilin He, Chaoyun Zhang, Qingwei Lin, Dan Pei, Pinjia He, Dongmei Zhang, and Qi Zhang. 2025. OpenRCA: Can Large Language Models Locate the Root Cause of Software Failures?. In *Proceedings of the 13th International Conference on Learning Representations (ICLR)*.

[39] Zhenhe Yao, Haowei Ye, Changhua Pei, Guang Cheng, Guangpei Wang, Zhiwei Liu, Hongwei Chen, Hang Cui, Zeyan Li, Jianhui Li, Gaogang Xie, and Dan Pei. 2024. SparseRCA: Unsupervised Root Cause Analysis in Sparse Microservice Testing Traces. In *Proceedings of the 35th IEEE International Symposium on Software Reliability Engineering (ISSRE)*. 391–402.

[40] Guangba Yu, Pengfei Chen, Hongyang Chen, Zijie Guan, Zicheng Huang, Linxiao Jing, Tianjun Weng, Xinmeng Sun, and Xiaoyun Li. 2021. MicroRank: End-to-End Latency Issue Localization with Extended Spectrum Analysis in Microservice Environments. In *Proceedings of the Web Conference 2021 (WWW '21)*. ACM, 3087–3098.

[41] Chenxi Zhang, Zhen Dong, Xin Peng, Bicheng Zhang, and Miao Chen. 2024. Trace-based Multi-Dimensional Root Cause Localization of Performance Issues in Microservice Systems. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*.

[42] Songhan Zhang, Aoyang Fang, Yifan Yang, Ruiyi Cheng, Xiaoying Tang, and Pinjia He. 2025. DynaCausal: Dynamic Causality-Aware Root Cause Analysis for Distributed Microservices. *arXiv preprint arXiv:2510.22613* (2025).

[43] Shenglin Zhang, Xiaoyu Feng, Runzhou Wang, Minghua Ma, Wenwei Gu, Yongqian Sun, Zedong Jia, Jinrui Sun, and Dan Pei. 2025. Too Many Cooks: Assessing the Need for Multi-Source Data in Microservice Failure Diagnosis. In *Proceedings of the 36th IEEE International Symposium on Software Reliability Engineering (ISSRE)*.

[44] Wei Zhang, Hongcheng Guo, Jian Yang, Zhoujin Tian, Yi Zhang, Chaoran Yan, Zhoujun Li, Tongliang Li, Xu Shi, Liangfan Zheng, and Bo Zhang. 2024. mABC: Multi-Agent Blockchain-inspired Collaboration for Root Cause Analysis in Micro-Services Architecture. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. Association for Computational Linguistics, 4017–4033.

[45] Ziming Zhao, Zhenwei Wang, Tiehua Zhang, Zhishu Shen, Hai Dong, Zhen Lei, Xingjun Ma, Gaowei Xu, Zhijun Ding, and Yun Yang. 2024. CHASE: A Causal Hypergraph based Framework for Root Cause Analysis in Multimodal Microservice Systems. *arXiv preprint arXiv:2406.19711* (2024).

[46] Lecheng Zheng, Zhengzhang Chen, and Haifeng Chen. 2025. Online Multi-Modal Root Cause Identification in Microservice Systems. In *Proceedings of the 2025 IEEE International Conference on Big Data (BigData)*. 1661–1668.

[47] Lecheng Zheng, Zhengzhang Chen, Jingrui He, and Haifeng Chen. 2024. MULAN: Multi-modal Causal Structure Learning and Root Cause Analysis for Microservice Systems. In *Proceedings of the ACM Web Conference 2024 (WWW)*. 4107–4116.

[48] Lecheng Zheng, Zhengzhang Chen, Dongjie Wang, Chengyuan Deng, Reon Matsuoka, and Haifeng Chen. 2024. LEMMA-RCA: A Large Multi-modal Multi-domain Dataset for Root Cause Analysis. *arXiv preprint arXiv:2406.05375* (2024).

[49] Xiang Zhou, Xin Peng, Tao Xie, Jun Sun, Chao Ji, Wenhai Li, and Dan Ding. 2021. Fault Analysis and Debugging of Microservice Systems: Industrial Survey, Benchmark System, and Empirical Study. *IEEE Transactions on Software Engineering* 47, 2 (2021), 243–260.

[50] Thomas Zimmermann, Nachiappan Nagappan, Harald Gall, Emanuel Giger, and Brendan Murphy. 2009. Cross-project Defect Prediction: A Large Scale Experiment on Data vs. Domain vs. Process. In *Proceedings of the 7th Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering (ESEC/FSE)*. 91–100.